

Weather Datenmodell mit POJO umsetzen

Einleitung:

POJO ist eine Abkürzung für Plain Old Java Object, also ein „ganz normales“ Objekt in der Programmiersprache Java.

In der Regel befinden sich die Daten in einem Java Projekt im Datenmodell. Das Datenmodell ist aus mehreren "ganz normalen" Java Klassen und deren Instanzen also POJOs aufgebaut.

Mit den *CodecProvider* bietet MongoDB eine sehr einfache Möglichkeit, dieses Datenmodell aus POJOs in die NoSQL Datenbank zu schreiben. Es entstehen dabei automatisch *key-value pairs*, *emdedded objects/ embedded documents und arrays*

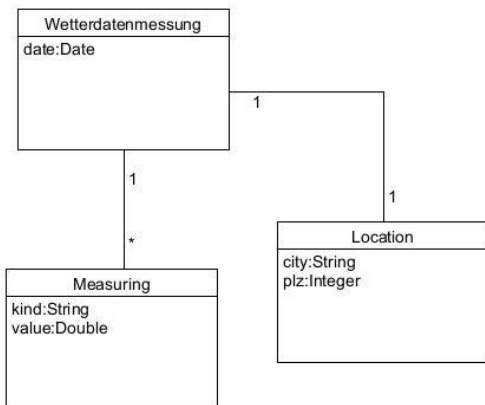
Ziel

- ⇒ Sie können eine Verbindung auf eine MongoDB herstellen.
- ⇒ Sie können einer Verbindung auf eine Collection herstellen.
- ⇒ Sie können neue Dokumente mit Hilfe von CodecProvider direkt aus POJOs erstellen.

Inhalte

Einleitung:	1
Ziel	1
Inhalte	1
Ausgangslage Datenmodell	2
Aufgabe Basisprojekt übernehmen und Projekt zum Laufen bringen	2
Aufgabe Datenklassen ergänzen.	3
Tipp: lombok verwenden, Getter&Setter sparen.	4
Aufgabe: Daten-Objekte aufbauen	4
Aufgabe: Den CodecProvider einsetzen.....	5
Bemerkung zu POJO und CodecProvider	5
Zusatzaufgabe Zugriff auf Datenbank kapseln	5

Ausgangslage Datenmodell



```

1 //Wetter Messung
2 {
3     "type": "Wettermessung",
4     "timestamp": "2022-03-11:12.00.00",
5     "station": { "city": "Winterthur", "plz": "8400" },
6     "measures": [
7         { "kind": "temperature", "value": 20.1 },
8         { "kind": "windspeed", "value": 2.3 }
9     ]
10 },
11 {
12     "type": "Wettermessung",
13     "timestamp": "2022-04-12:11.00.00",
14     "station": { "city": "Zürich", "plz": "8004" },
15     "measures": [
16         { "kind": "rain", "value": 0.0 },
17         { "kind": "temperature", "value": 18.3 }
18     ]
19 }
  
```

Aufgabe Basisprojekt übernehmen und Projekt zum Laufen bringen

- Übernehmen Sie das Basis-Projekt.
- Sie brauchen dazu eine *Mongo-Datenbankserver* und *MongoExpress* (am einfachsten beides als Dockercontainer) und allenfalls *MongoCompass*.
- Bringen Sie das Projekt zum Laufen.
- Analysieren Sie, was bereits vorhanden ist.

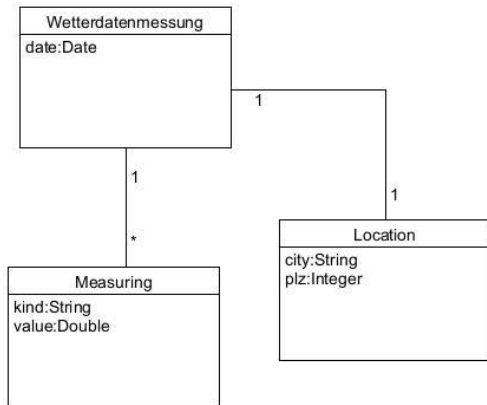
```

24 public static void main(String[] args) {
25     System.out.println("Hello Weather");
26
27     //connect to db
28     String connectionString = "mongodb://root:1234@localhost:27017";
29     MongoClient mongoClient = MongoClient.create(connectionString);
30
31     //list all databases
32     System.out.println("List all databases:");
33     mongoClient.listDatabases().forEach((Consumer<? super Document>) result -> System.out.println(result.toJson()));
34
35     //close connection
36     mongoClient.close();
37 }
38 }
  
```

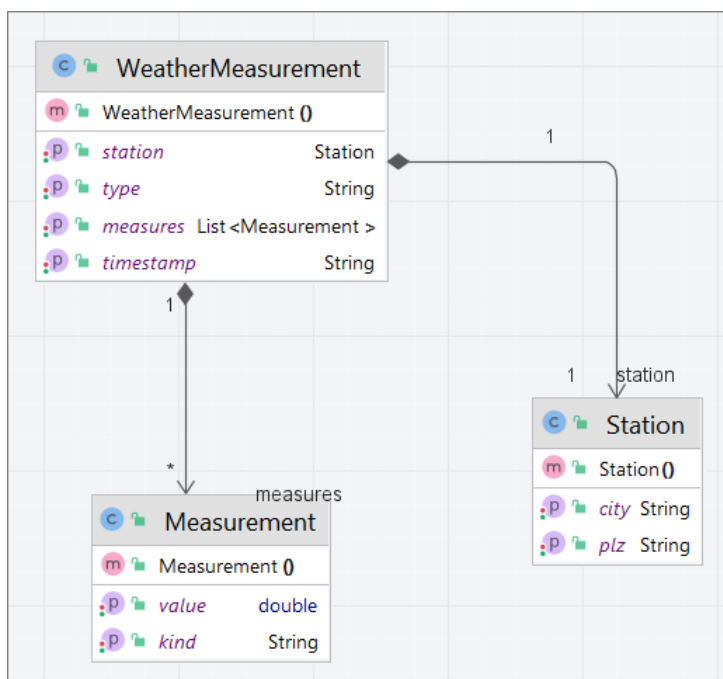
Aufgabe Datenklassen ergänzen.

Zunächst müssen Sie das konzeptionelle Modell bereinigen.

Damit die Assoziationen erfüllt sind, muss *WeatherMeasurement* ein *Attribut für die Location* sowie eine *Liste von Measurement* haben.



Daraus abgeleitete Klassen



Aufgabe:

- Ergänzen Sie im Projekt ein *package ch bbw.pr.weather.model* und im Package die Datenklassen gemäss dem obigen Modell.

Bevor Sie Getter und Setter implementieren, lesen Sie auf der nächsten Seite weiter!

Tipp: lombok verwenden, Getter&Setter sparen.

Wenn Sie die Bibliothek *lombok* verwenden, müssen Sie keine Getter und Setter in den Datenklassen implementieren.

- Ergänzen Sie zunächst im pom.xml diese Dependency und laden Sie das pom.xml neu.

```

53      <!-- https://mvnrepository.com/artifact/org.projectlombok/lombok -->
54      <dependency>
55          <groupId>org.projectlombok</groupId>
56          <artifactId>lombok</artifactId>
57          <version>1.18.24</version>
58          <scope>provided</scope>
59      </dependency>
  
```

Sie können Sie lombok in Datenklassen anwenden, hier das Beispiel einer Klasse *Station*:

```

1  package ch.bbw.pr.weather.model;
2  import lombok.Getter;
3  import lombok.Setter;
4
5  /**
6   * @author Peter Rutschmann
7   * @version 18.11.2022
8   */
9  @Getter
10 @Setter
11 public class Station {
12     private String city;
13     private String plz;
14 }
  
```

Aufgabe:

- Wenden Sie lombok in Ihren drei Datenklassen an.

Aufgabe: Daten-Objekte aufbauen

- Instanziiieren Sie nun die Daten Objekte und befüllen Sie die Attribute mit Daten.

Wenden Sie die Daten anhand dieses Beispiels an.

- weatherMeasurement:
 type → "Wettermessung"
 timestamp → new Date().toString()
 station:
 city → "Winterthur"
 plz → "8400"
 measures:
 [0]
 kind -> temperature
 value -> 20.1
 [1]
 kind -> windspeed
 value -> 2.3

Aufgabe: Den CodecProvider einsetzen

Damit Sie die Datenobjekte direkt in die MongoDB schreiben können, brauchen Sie einen *CodecProvider* für das *package ch.bbw.pr.weather.model*.

Fügen Sie in `main()` *CodecProvider* der *CodecRegistry* hinzu.

```
CodecProvider.pojoCodecProvider =  
    PojoCodecProvider.builder().register("ch.bbw.pr.weather.model").build();  
  
CodecRegistry.pojoCodecRegistry =  
    fromRegistries(com.mongodb.MongoClient.getDefaultCodecRegistry(),  
        fromProviders(pojoCodecProvider));
```

Dann müssen Sie die *statisticCollection* mit der *CodeRegistry* bekannt machen

```
statisticCollection =  
    statisticCollection.withCodecRegistry(pojoCodecRegistry);
```

Und mit wenigen Zeilen können Sie nun das Document erstellen:

```
//transform pojo to document  
Document doc = new Document();  
doc.append("weatherMeasurement", weatherMeasurement);
```

Aufgabe:

- Setzen Sie das Obige um.
- Kontrollieren Sie das Ergebnis in der Datenbank.

Bemerkung zu POJO und CodecProvider

Bei diesem Vorgehen liegt der Aufwand eindeutig mehr bei den Klassen und den Daten-Objekten.

Das Schreiben in das Document ist hingegen sehr einfach.

Ein Java Projekt arbeitet in der Regel immer mit einem oder mehreren Modellen. Das heisst die Klassen und die Objekte mit Daten existieren, da sie vom Java Projekt bereits benötigt werden.

Nun ein Datenmodell oder einen Teil davon in eine NoSQL Datenbank zu schreiben wird mit dem CodeProvider sehr einfach.

Für diesen Anwendungsfall ist das ein geeigneter Weg.

Kann man nun aus dem Document über diesen Weg die Daten-Objekte auch wieder herstellen?

Hier stellt sich eher die Frage für was?

Sind NoSQL Datenbanken nicht eher im Einsatz, wenn man aus den Daten in der Datenbank etwas Neues z.B. eine Auswertung erstellt?

Und sind Sie damit weniger gedacht, um die ursprünglichen Daten wieder als Objekte abzubilden?

Zusatzaufgabe Zugriff auf Datenbank kapseln

Kapseln Sie den Zugriff auf die Datenbank in einer eigenen Klasse.